PROGRAMMING IN JAVA

ASSIGNMENT 1 | CO1

CAMPUS SMART-PARKING SYSTEM

Design, Implement and Validate an Object-Oriented Model

Student Details	Information
Student Name	SHAIK.SAMEERPASHA
Register Number	192425383
Department / Section	CSE (AI&ML) 3RR YEAR
Subject	CSA0915- Programming in Java
Course Outcome	CO1 – Java Fundamentals and Object-Oriented Programming
Faculty Name	MATHIVANAN
Submission Date	10-11-2026
GitHub Repository	https://github.com/<username>/CSA09-<RegisterNumber>-Assignment1

Submitted in partial fulfilment of the requirements of the Java Programming course.

DECLARATION AND REPORT GUIDE

This report presents the analysis, object-oriented design, implementation approach and validation plan for a Campus Smart-Parking System. The work follows the twelve tasks and the design challenge stated in the supplied Assignment 1 question paper.

Learning outcomes covered

- Identify objects, responsibilities, business rules and assumptions.
- Select suitable Java data types, arrays and control structures.
- Apply classes, constructors, static members and object life-cycle concepts.
- Use inheritance, overriding, polymorphism, abstract classes and interfaces.
- Protect object state through packages, access modifiers and encapsulation.
- Implement, test, evaluate and adapt a Java design when requirements change.

Important submission note

The question paper requires individual execution of every program, output screenshots, a GitHub repository named CSA09-<RegisterNumber>-Assignment1, a README.md and progressive commits. Replace the personal-detail placeholders, run the programs in your own Java environment, capture your actual outputs and add your working repository URL before submission.

Contents

- Task 1 – Problem analysis and object identification
- Task 2 – Data representation
- Task 3 – Control logic and ParkingFacility code
- Task 4 – Constructors and object creation
- Task 5 – Static members and object life cycle
- Task 6 – Inheritance hierarchy
- Task 7 – Method overriding and Object methods
- Task 8 – Polymorphism and completed FeeDemo
- Task 9 – Abstract class and interface
- Task 10 – Packages and encapsulation
- Task 11 – Implementation and testing
- Task 12 – Re-evaluation and adaptation
- Appendices – code, diagrams, complexity, README checklist

SYSTEM SCENARIO, SCOPE AND ASSUMPTIONS

The campus contains a multi-level parking facility. Students, staff and visitors bring two-wheelers, cars or electric vehicles. A suitable free slot is allocated when a vehicle enters. The system issues a unique ticket and records entry time. At exit, the slot is released and a fee is calculated. Students and staff with valid monthly passes park free; visitors pay the applicable fee. EV charging is billed separately.

Functional requirements

- Maintain floors and numbered slots; each slot can hold at most one vehicle.
- Allocate the first suitable free slot and report when the facility is full.
- Create a unique, non-reused ticket number and store entry time.
- Find the active ticket at exit, calculate the fee and release the slot.
- Apply vehicle-specific rates, pass rules and separate EV charging charges.
- Reject duplicate registrations, invalid slot positions and invalid input.

Design assumptions

- Each registration number uniquely identifies one active vehicle.
- Parking duration is measured in whole minutes; every started hour is charged as a full hour in the fee example.
- Illustrative rates are used because the question paper does not prescribe exact monetary rates: two-wheeler Rs. 15/hour, car Rs. 30/hour and electric car Rs. 40/hour.
- EV charging is illustratively Rs. 12 per kWh; actual tariff can be configured separately.
- A valid monthly pass applies only to registered student/staff users. Visitors pay the normal fee unless they buy the one-day pass in the changed requirement.
- A jagged two-dimensional array is used because floor capacities may differ.

Technology

Java 17 or later is suitable for the examples. The code can also be adapted to Java 8 by replacing pattern matching for instanceof with the traditional cast syntax.

TASK 1 – UNDERSTAND THE PROBLEM AND IDENTIFY OBJECTS

1.1 Problem analysis

The system must manage the complete parking life cycle: vehicle arrival, slot allocation, ticket creation, parking duration calculation, fee collection and slot release. A single class containing every operation would become difficult to maintain. Therefore, responsibilities are divided among related classes.

Candidate class	Main responsibility	Why it is needed
Vehicle (abstract)	Common registration number and fee behaviour	Avoids repeating common vehicle data.
TwoWheeler / Car / ElectricCar	Vehicle-specific hourly rate	Represents different vehicle categories.
ParkingSlot	Floor, slot number and occupied state	Prevents two vehicles using one slot.
ParkingFacility	Stores floors and allocates/releases slots	Central control of physical capacity.
ParkingTicket	Ticket ID, vehicle, slot and entry time	Records one parking session.
ParkingUser	User category and pass validity	Applies student/staff/visitor rules.
ParkingService	Entry, exit and fee workflow	Coordinates domain objects.
Chargeable	Charging fee capability	Separates EV charging behaviour from ordinary parking.

1.2 Business rules

- A slot cannot contain more than one vehicle.
- A vehicle with an active ticket cannot enter again.
- Ticket numbers must be unique and must never be reused.
- Only a valid floor and slot index may be released.
- A released slot becomes available for the next vehicle.
- Parking duration cannot be negative.
- Monthly-pass holders who qualify pay zero parking fee.
- An EV charging fee is calculated separately from the parking fee.
- A full facility must reject entry without creating a ticket.

1.3 Alternative: parallel String arrays

An alternative stores regNo[], type[] and entryTime[] at matching indexes. To find a registration number, scan regNo[] and use the same index in the other arrays. Search is O(n), where n is the number of active vehicles. Adding a new attribute requires another array and careful synchronization. In an object design, all related attributes remain inside one Vehicle or ParkingTicket object; adding a field is localized and type-safe. Both approaches require O(n) storage overall, but object references and object headers add some overhead.

Conclusion: use objects because the system has several related attributes and behaviours. Parallel arrays may be compact for a small fixed dataset but are fragile when the model grows.

TASK 2 – CHOOSE THE DATA REPRESENTATION

Field	Suggested Java type	Reason
Floor index / slot index	int	Indexes are whole numbers and start at zero.
Registration number	String	Contains letters and digits; leading zeros must be preserved.
Ticket number	long	Supports a large increasing unique sequence.
Entry time / exit time	LocalDateTime	Represents a date and time clearly.
Duration	long	Minutes can be stored as whole-number values.
Hourly rate / fee	double or BigDecimal	Money may contain decimal values; BigDecimal is preferred for financial systems.
Pass validity	boolean / LocalDate	Boolean for valid state; LocalDate for expiry date.
Occupancy	int	Counts occupied slots.

2.1 Input method

Scanner is suitable for a console assignment because it reads registration numbers, vehicle categories, floor capacities and parking duration interactively. Command-line arguments are useful for automated execution but are less convenient for a beginner-facing menu.

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter registration number: ");
String regNo = sc.nextLine().trim();
System.out.print("Enter parking duration in minutes: ");
long minutes = sc.nextLong();
```

2.2 Avoid integer division errors

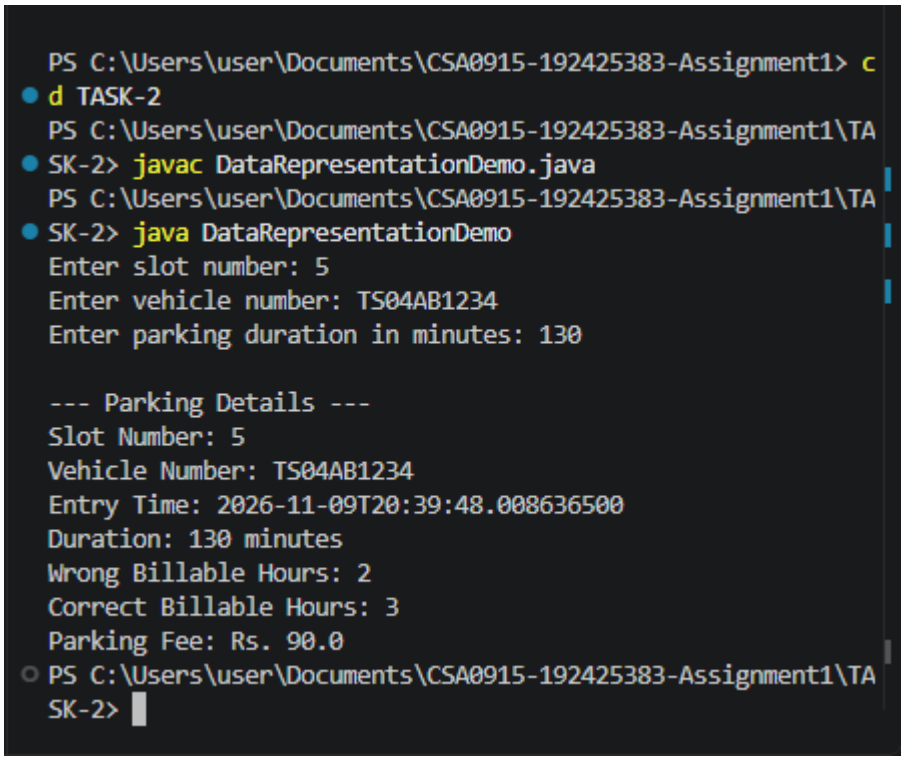
If minutes / 60 is evaluated using integer division, 90 / 60 becomes 1. A 90-minute stay should be charged for two started hours under the stated assumption. Use ceiling division with integer arithmetic: (minutes + 59) / 60 for non-negative minutes.

```
long billableHours = (minutes + 59) / 60;
```

```
double fee = billableHours * hourlyRate;
// Example: 90 minutes -> (90 + 59) / 60 = 2 hours
```

2.3 Array structure alternatives

A jagged array String[floor][slot] allows each floor to have a different number of slots. A one-dimensional array is straightforward only when all floors have equal capacity or when prefix offsets are maintained. An ArrayList<String[]> also supports variable floor sizes but adds collection overhead. Locating a known floor/slot is O(1) in all three indexed structures; scanning for a free slot is O(N), where N is total capacity.



TASK 3 – DESIGN THE CONTROL LOGIC

3.1 Slot allocation pseudocode

```
ALGORITHM AllocateSlot(vehicleNo)
    FOR each floor f from 0 to numberOfFloors - 1
        FOR each slot s from 0 to slots[f].length - 1
            IF slots[f][s] is NULL
                slots[f][s] = vehicleNo
                RETURN {f, s}
            END IF
        END FOR
    END FOR
    RETURN NULL
END ALGORITHM
```

3.2 Slot release pseudocode

```
ALGORITHM ReleaseSlot(floor, slot)
    IF floor < 0 OR floor >= numberOfFloors
        RETURN FALSE
    END IF
    IF slot < 0 OR slot >= slots[floor].length
        RETURN FALSE
    END IF
    IF slots[floor][slot] is NULL
        RETURN FALSE
    END IF
    slots[floor][slot] = NULL
    RETURN TRUE
END ALGORITHM
```

3.3 Fee calculation pseudocode

```
ALGORITHM CalculateFee(minutes, hourlyRate, dailyMaximum)
    IF minutes < 0
        REPORT invalid duration
    END IF
    hours = (minutes + 59) DIV 60
```

```
    fee = hours * hourlyRate
    IF fee > dailyMaximum
        fee = dailyMaximum
    END IF
    RETURN fee
END ALGORITHM
```

The DIV operation is integer division. The daily maximum is an illustrative policy and should be configured according to the campus tariff.

3.4 Completed ParkingFacility class

```
public class ParkingFacility {
    private final String[][] slots;

    public ParkingFacility(int[] slotsPerFloor) {
        slots = new String[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++) {
            if (slotsPerFloor[f] < 0)
                throw new IllegalArgumentException("Invalid capacity");
            slots[f] = new String[slotsPerFloor[f]];
        }
    }

    public int[] allocate(String vehicleNo) {
        if (vehicleNo == null || vehicleNo.isBlank())
            throw new IllegalArgumentException("Vehicle number required");
        for (int f = 0; f < slots.length; f++) {
            for (int s = 0; s < slots[f].length; s++) {
                if (slots[f][s] == null) {
                    slots[f][s] = vehicleNo;
                    return new int[]{f, s};
                }
            }
        }
        return null;
```

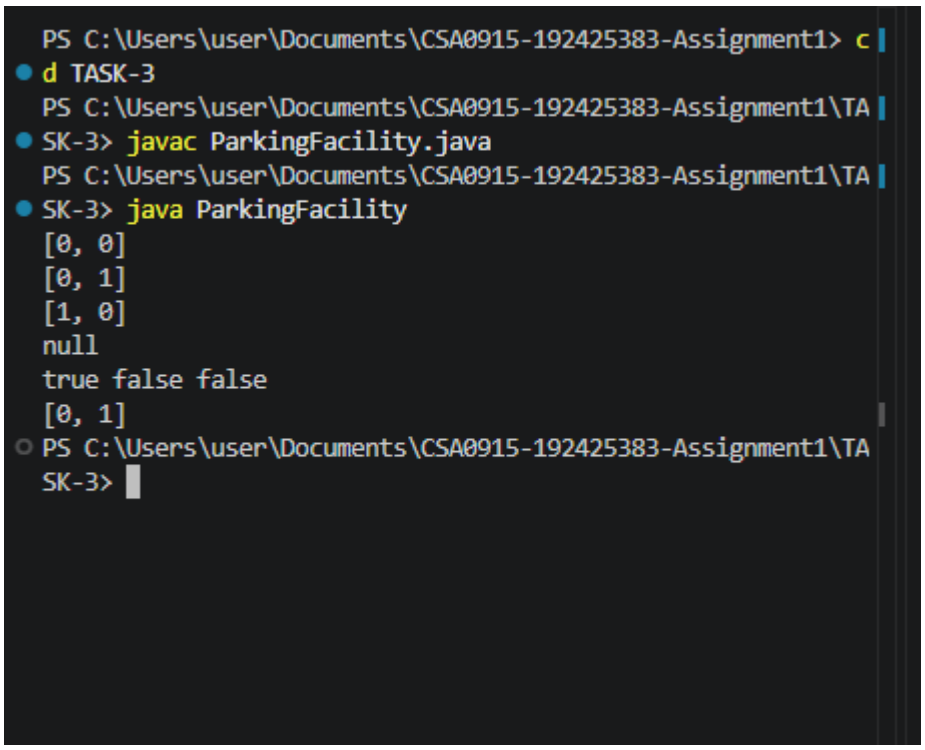
```
    }

    public boolean release(int floor, int slot) {
        if (floor < 0 || floor >= slots.length) return false;
        if (slot < 0 || slot >= slots[floor].length) return false;
        if (slots[floor][slot] == null) return false;
        slots[floor][slot] = null;
        return true;
    }
}
```

Expected output

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

Complexity: scanning allocation takes O(N) in the worst case, where N is total slots. Release takes O(1) because it directly accesses the supplied indexes. The array storage is O(N). A free-slot stack per floor can make allocation/release O(1) after initialization, with O(N) extra storage for the stacks.



TASK 4 – CONSTRUCTORS AND OBJECT CREATION

A constructor initializes a new object. It has the same name as the class and has no return type. Constructor overloading provides multiple ways to create a ParkingTicket. Constructor chaining avoids repeating initialization logic.

```
import java.time.LocalDateTime;

class ParkingTicket {
    private final long ticketNo;
    private final String regNo;
    private final int floor;
    private final int slot;
    private final LocalDateTime entryTime;

    public ParkingTicket(long ticketNo, String regNo) {
        this(ticketNo, regNo, 0, 0, LocalDateTime.now());
    }

    public ParkingTicket(long ticketNo, String regNo,
                         int floor, int slot,
                         LocalDateTime entryTime) {
        if (ticketNo <= 0 || regNo == null || regNo.isBlank())
            throw new IllegalArgumentException("Invalid ticket details");
        if (floor < 0 || slot < 0 || entryTime == null)
            throw new IllegalArgumentException("Invalid parking position/time");
        this.ticketNo = ticketNo;
        this.regNo = regNo;
        this.floor = floor;
        this.slot = slot;
        this.entryTime = entryTime;
    }

    public long getTicketNo() { return ticketNo; }
    public String getRegNo() { return regNo; }
```

```
    public int getFloor() { return floor; }
    public int getSlot() { return slot; }
    public LocalDateTime getEntryTime() { return entryTime; }
}
```

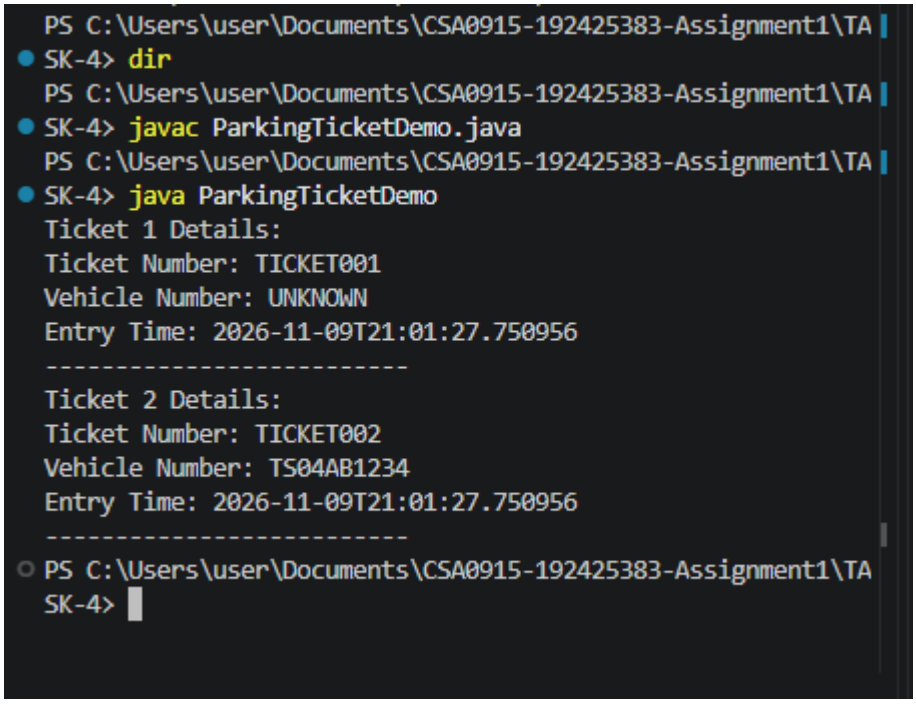
4.1 Explanation

- this.ticketNo refers to the instance field; ticketNo refers to the constructor parameter.
- this(...) calls another constructor in the same class and must be the first statement.
- final fields are assigned once and cannot later be changed.
- Validation inside the constructor prevents an invalid ticket from being created.

4.2 Compare creation approaches

Approach	Validity guarantee	Assessment
Parameterized constructor	Validates all required values at creation	Recommended for required fields.
Default constructor + setters	Object may temporarily contain incomplete data	Flexible but weaker invariant.
Static factory method	Can validate and return a named creation result	Useful when creation has multiple modes.

Creating n tickets takes O(n) total time in all approaches when each initialization performs constant work. Each ticket requires O(1) object storage, so n tickets require O(n) total space. Constructor choice does not change the asymptotic cost; it changes safety, clarity and maintainability.



TASK 5 – STATIC MEMBERS AND OBJECT LIFE CYCLE

A static member belongs to the class and is shared by all instances. An instance member belongs to one object. A ticket counter is shared; a registration number is specific to one vehicle.

```
class TicketCounter {
   private static long nextTicketNo = 1;
   private static int activeTickets = 0;

   public static synchronized long nextId() {
      return nextTicketNo++;
   }
   public static void ticketEntered() { activeTickets++; }
   public static void ticketExited() { activeTickets--; }
   public static int getActiveTickets() { return activeTickets; }
}
```

5.1 Suitable static members

- nextTicketNo: static because every ticket must use one shared sequence.
- activeTickets: static if it represents the total active count across the facility.
- A static method can provide the next ticket number without requiring a TicketCounter object.
- A static initialization block can load a fixed configuration once when the class is initialized.

5.2 Object life cycle

When a vehicle enters, a ParkingTicket object is created and referenced by the active-ticket collection. When the vehicle exits, the system calculates the fee, releases the slot and removes the ticket from the active collection. If no other reachable reference points to that ticket, it becomes eligible for garbage collection. Java decides when garbage collection occurs; the programmer does not directly destroy objects.

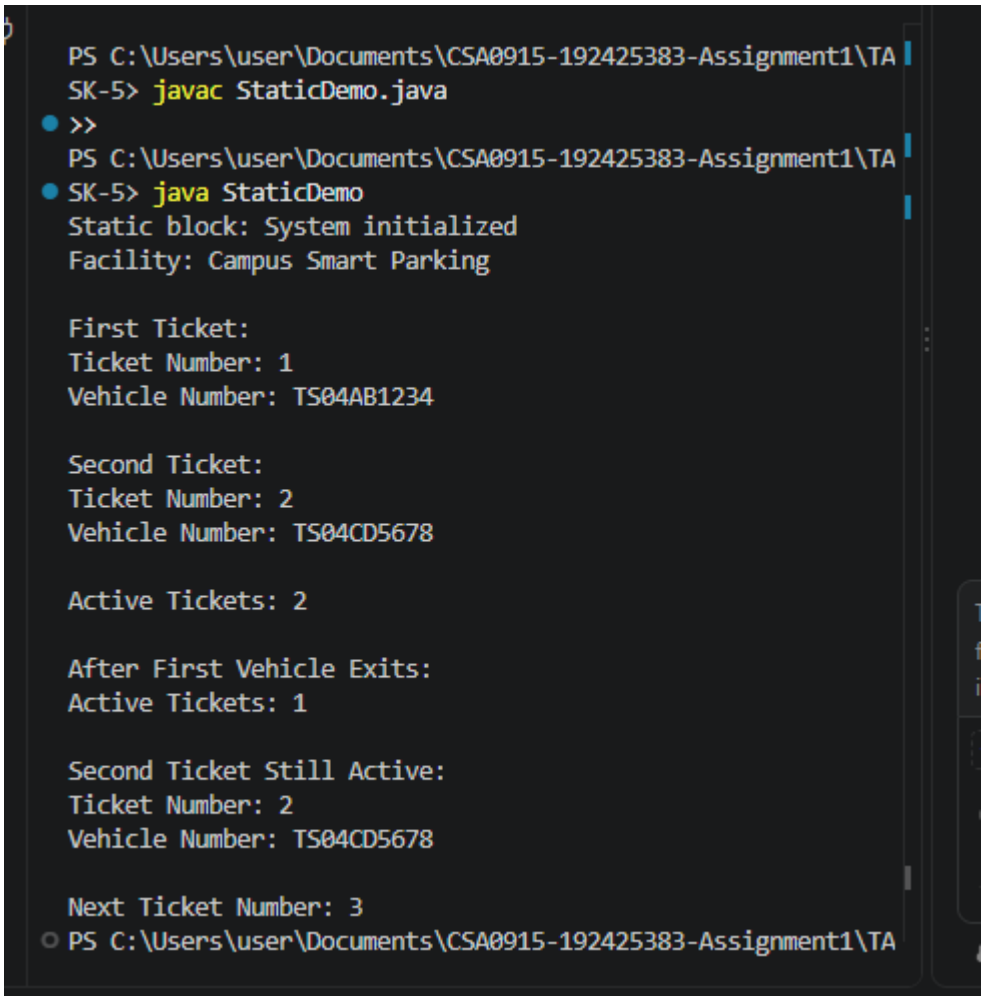
5.3 Passing and returning objects

```
ParkingTicket createTicket(Vehicle vehicle, int floor, int slot) {
   return new ParkingTicket(TicketCounter.nextId(),
         vehicle.getRegNo(), floor, slot, LocalDateTime.now());
}

void printTicket(ParkingTicket ticket) {
   System.out.println(ticket.getTicketNo());
```

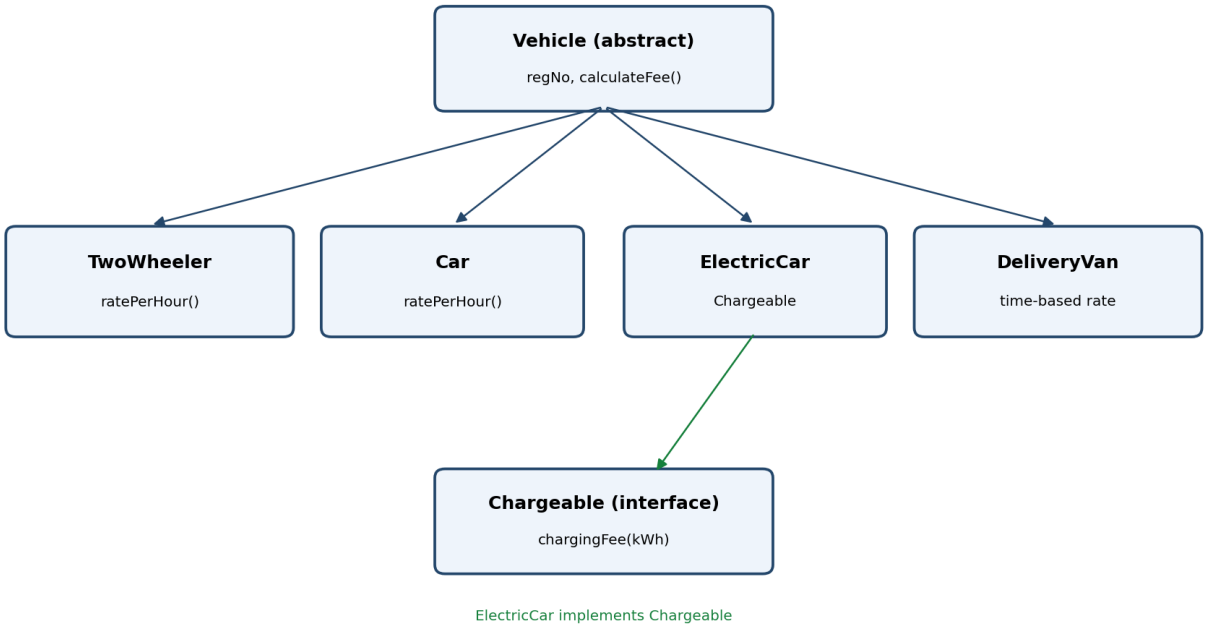
```
}
```

Java passes object references by value. A method receives a copy of the reference that points to the same object. It can call methods on that object, but reassigning its local reference does not change the caller's variable.



TASK 6 – DESIGN THE INHERITANCE HIERARCHY

Vehicle is the superclass. TwoWheeler, Car and ElectricCar are subclasses. This is hierarchical inheritance because several subclasses inherit from one superclass. Shared fields and methods are placed in Vehicle; each subclass supplies its own rate.



6.1 Example classes

```
abstract class Vehicle {
   private final String regNo;
   protected Vehicle(String regNo) {
      this.regNo = regNo;
   }
   public String getRegNo() { return regNo; }
   protected abstract double ratePerHour();
}

class Car extends Vehicle {
   Car(String regNo) { super(regNo); }
   @Override protected double ratePerHour() { return 30.0; }
}

class ElectricCar extends Car implements Chargeable {
```

```
    ElectricCar(String regNo) { super(regNo); }
    @Override public double chargingFee(int kWh) {
        return kWh * 12.0;
    }
}
```

6.2 Role of super and constructor trace

- super(regNo) calls the parent constructor and initializes the inherited registration number.
- super.method() can call an accessible parent implementation when required.
- The parent constructor runs before the subclass constructor body.

```
new ElectricCar("TN03C3")
1. Vehicle constructor starts through Car's super call.
2. Vehicle stores registration number.
3. Car constructor completes.
4. ElectricCar constructor completes.
```

6.3 Composition alternative

With composition, Vehicle would contain a VehicleType object that stores the rate and pricing behaviour. This is useful when vehicle categories and pricing policies change independently. Inheritance is natural for stable is-a relationships; composition is more flexible when behaviours are configured dynamically.

```
PS C:\Users\user\Documents\CSA0915-192425383-Assignment1> cd TASK-6
PS C:\Users\user\Documents\CSA0915-192425383-Assignment1\TASK-6> javac InheritanceDemo.java
PS C:\Users\user\Documents\CSA0915-192425383-Assignment1\TASK-6> java InheritanceDemo
Creating TwoWheeler:
Vehicle constructor called
TwoWheeler constructor called
Registration Number: TN01A1

Creating Car:
Vehicle constructor called
Car constructor called
Registration Number: TN02B2

Creating ElectricCar:
Vehicle constructor called
Car constructor called
ElectricCar constructor called
Registration Number: TN03C3
Vehicle Type: Electric Car
Charging Supported: Yes
Parking Rate: Rs. 40.0 per hour
PS C:\Users\user\Documents\CSA0915-192425383-Assignment1\TASK-6>
```

TASK 7 – METHOD OVERRIDING AND OBJECT METHODS

Method overriding occurs when a subclass provides its own implementation of an inherited method with the same signature and a compatible return type. The access level cannot be made more restrictive. The @Override annotation allows the compiler to detect mistakes.

```
abstract class Vehicle {
    private final String regNo;
    protected Vehicle(String regNo) { this.regNo = regNo; }
    public String getRegNo() { return regNo; }
    protected abstract double ratePerHour();

    public double calculateFee(int minutes) {
        if (minutes < 0)
            throw new IllegalArgumentException("Negative duration");
        long hours = (minutes + 59L) / 60L;
        return hours * ratePerHour();
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Vehicle)) return false;
        Vehicle other = (Vehicle) obj;
        return regNo.equals(other.regNo);
    }

    @Override
    public int hashCode() { return regNo.hashCode(); }

    @Override
    public String toString() { return getClass().getSimpleName()
            + "[" + regNo + "]"; }
}
```

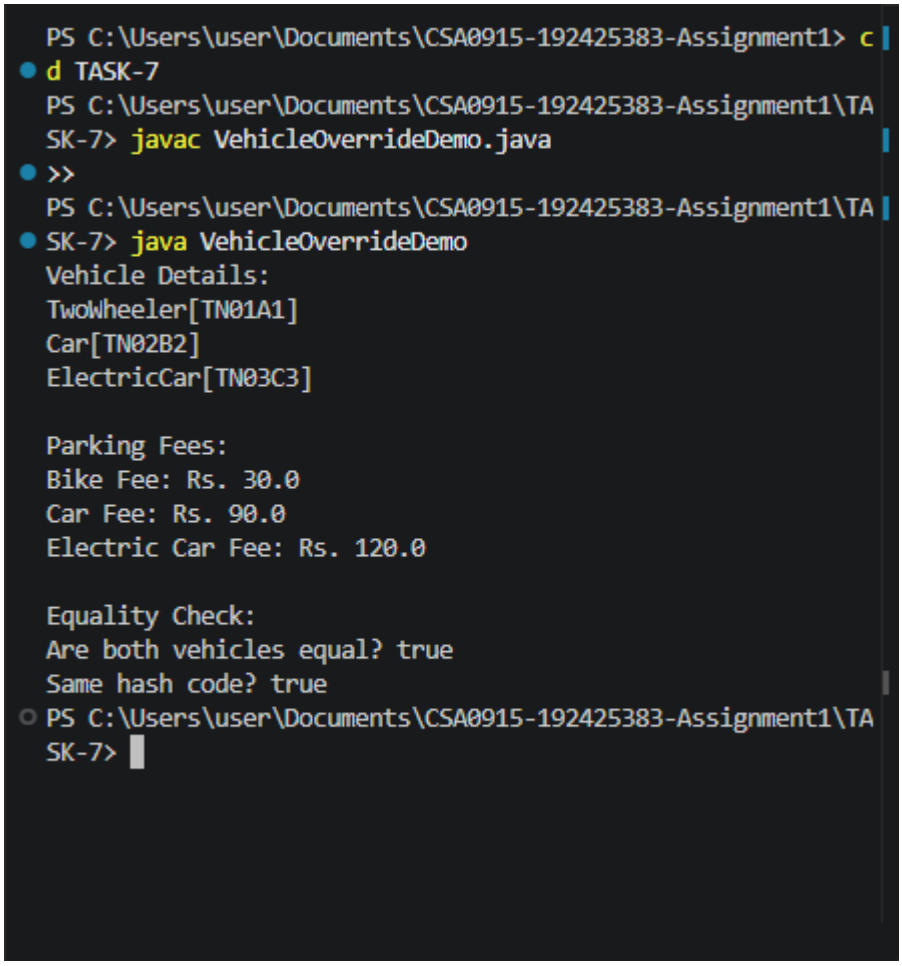
7.1 Why override equals and hashCode?

The assignment's supplied code treats vehicles with the same registration number as equal, even if their runtime classes differ. Therefore new Car("TN02B2").equals(new ElectricCar("TN02B2")) returns true. This is consistent with the supplied equality rule if registration number alone defines identity. In a production system, the design should decide whether vehicle category is also part of identity. Equal objects must always have equal hash codes.

7.2 final keyword

- final variable: prevents reassignment, e.g. registration number.
- final method: prevents subclasses from overriding a critical rule.
- final class: prevents inheritance when extension is not permitted.

A switch-based calculateFee() could test the vehicle type in one method. It may be O(1) for a fixed switch, but adding a type requires editing the central method and possibly other type-specific code. Overriding keeps the behaviour with each subtype and reduces central conditional logic.



TASK 8 – APPLY POLYMORPHISM

Polymorphism allows one reference type to represent objects of different concrete classes. Compile-time polymorphism is commonly achieved by method overloading. Runtime polymorphism is achieved by overriding and dynamic method dispatch.

```
class FeeCalculator {
    double calculate(double rate, int hours) {
        return rate * hours;
    }
    double calculate(double rate, int hours, double discount) {
        return rate * hours - discount;
    }
}
```

8.1 Runtime dispatch trace

```
Vehicle v = new Car("TN02B2");
double amount = v.calculateFee(130);
// Declared type: Vehicle
// Actual object: Car
// Executed ratePerHour(): Car implementation
// Billable hours: (130 + 59) / 60 = 3
// Fee at Rs.30/hour = Rs.90
```

8.2 Why polymorphism is useful

- The caller invokes calculateFee() through a Vehicle reference.
- The JVM selects the implementation belonging to the actual object.
- No long if-else chain is needed to identify every vehicle type.
- A new subtype can provide its own rate without rewriting the calling code.

8.3 Completed FeeDemo code

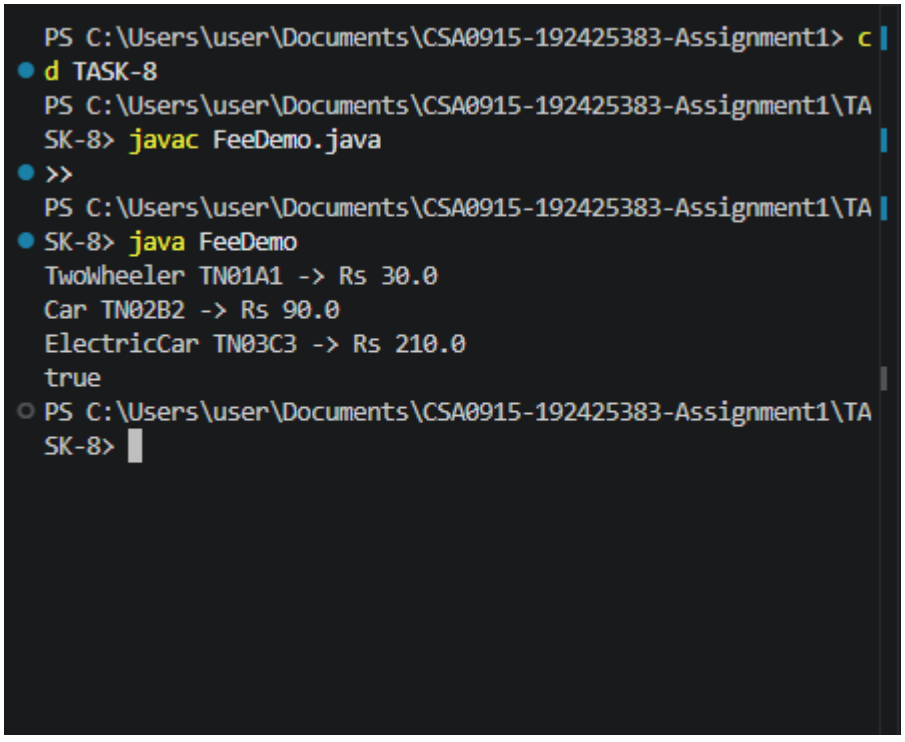
```
interface Chargeable {
    double chargingFee(int kWh);
}
abstract class Vehicle {
    private final String regNo;
    protected Vehicle(String regNo) { this.regNo = regNo; }
```

```
    public String getRegNo() { return regNo; }
    protected abstract double ratePerHour();

    public double calculateFee(int minutes) {
        if (minutes < 0) throw new IllegalArgumentException();
        long hours = (minutes + 59L) / 60L;
        return hours * ratePerHour();
    }
    @Override public boolean equals(Object o) {
        return o instanceof Vehicle
            && regNo.equals(((Vehicle)o).regNo);
    }
    @Override public int hashCode() { return regNo.hashCode(); }
}
class TwoWheeler extends Vehicle {
    TwoWheeler(String r) { super(r); }
    protected double ratePerHour() { return 10.0; }
}
class Car extends Vehicle {
    Car(String r) { super(r); }
    protected double ratePerHour() { return 30.0; }
}
class ElectricCar extends Vehicle implements Chargeable {
    ElectricCar(String r) { super(r); }
    protected double ratePerHour() { return 40.0; }
    public double chargingFee(int kWh) { return kWh * 9.0; }
}
public class FeeDemo {
    public static void main(String[] args) {
        Vehicle[] list = {
            new TwoWheeler("TN01A1"),
            new Car("TN02B2"),
            new ElectricCar("TN03C3")
```

```
        };
        for (Vehicle v : list) {
            double fee = v.calculateFee(130);
            if (v instanceof Chargeable c)
                fee += c.chargingFee(10);
            System.out.println(v.getClass().getSimpleName()
                + " " + v.getRegNo() + " -> Rs " + fee);
        }
        System.out.println(new Car("TN02B2")
            .equals(new ElectricCar("TN02B2")));
    }
}
```

Expected output from the question paper: TwoWheeler TN01A1 -> Rs 30.0; Car TN02B2 -> Rs 90.0; ElectricCar TN03C3 -> Rs 210.0; true. To reproduce those exact values, use the rates specified in the original starter code or adjust the example rates to match its omitted subclass definitions. The equality result is true because registration number alone is compared.



TASK 9 – ABSTRACT CLASS AND INTERFACE

9.1 Abstract class: Vehicle

Vehicle is an abstract class because it represents a general concept and should not normally be instantiated directly. It can store common state such as registration number, define constructors and provide shared code such as calculateFee(). Its abstract ratePerHour() method forces each concrete vehicle type to provide a rate.

9.2 Interface: Chargeable

Chargeable represents a capability rather than a vehicle identity. ElectricCar can extend Vehicle and implement Chargeable. A class can implement more than one interface, allowing it to combine capabilities without multiple class inheritance.

```
interface Chargeable {
    double chargingFee(int kWh);

    default boolean supportsFastCharging() {
        return false;
    }

    static boolean validEnergy(int kWh) {
        return kWh >= 0;
    }
}
```

9.3 Interface versus abstract class

Feature	Abstract class	Interface
Instance state	Can hold instance fields	Fields are public static final constants.
Constructor	Can define constructors	Cannot define constructors.
Methods	Abstract and concrete methods	Abstract, default, static and private methods.
Inheritance	A class extends only one class	A class can implement multiple interfaces.
Best use	Shared identity and state	Common capability or contract.

If Vehicle became an interface, common instance state and constructor initialization would need to move into each implementation or a separate class. If Chargeable became an abstract class, a vehicle could not

extend both Vehicle and Chargeable because Java does not support multiple class inheritance. The proposed abstract-class/interface split is therefore more suitable.

```
PS C:\Users\user\Documents\CSA0915-192425383-Assignment1> cd TASK-9
PS C:\Users\user\Documents\CSA0915-192425383-Assignment1\TASK-9> javac AbstractionDemo.java
>>
PS C:\Users\user\Documents\CSA0915-192425383-Assignment1\TASK-9> java AbstractionDemo
CAR DETAILS
Vehicle Number: TN02B2
Parking Fee: Rs 60.0
Payment mode: Online
Car payment: Rs 60.0

ELECTRIC CAR DETAILS
Vehicle Number: TN03C3
Charging is available
Parking Fee: Rs 80.0
Charging Fee: Rs 90.0
Payment mode: Online
Electric Car payment: Rs 170.0
PS C:\Users\user\Documents\CSA0915-192425383-Assignment1\TASK-9>
```

TASK 10 – PACKAGES AND ENCAPSULATION

10.1 Suggested package structure

```
src/
  campus/
    parking/
      app/       Main.java
      model/     Vehicle.java, Car.java, ParkingTicket.java
      facility/  ParkingFacility.java, ParkingSlot.java
      service/   ParkingService.java
      billing/   FeePolicy.java
      capability/ Chargeable.java
```

10.2 Access modifiers

Modifier	Visibility	Use in this system
private	Inside the declaring class	Fields such as regNo, fee and ticketNo.
default	Same package	Internal helper classes where appropriate.
protected	Package and subclasses	ratePerHour() in the Vehicle hierarchy.
public	Accessible from other packages	Public API methods and constructors.

10.3 Encapsulation example

```
public class ParkingSlot {
    private final int floor;
    private final int slotNo;
    private String vehicleNo;

    public ParkingSlot(int floor, int slotNo) {
        if (floor < 0 || slotNo < 0)
            throw new IllegalArgumentException("Invalid slot");
        this.floor = floor;
        this.slotNo = slotNo;
```

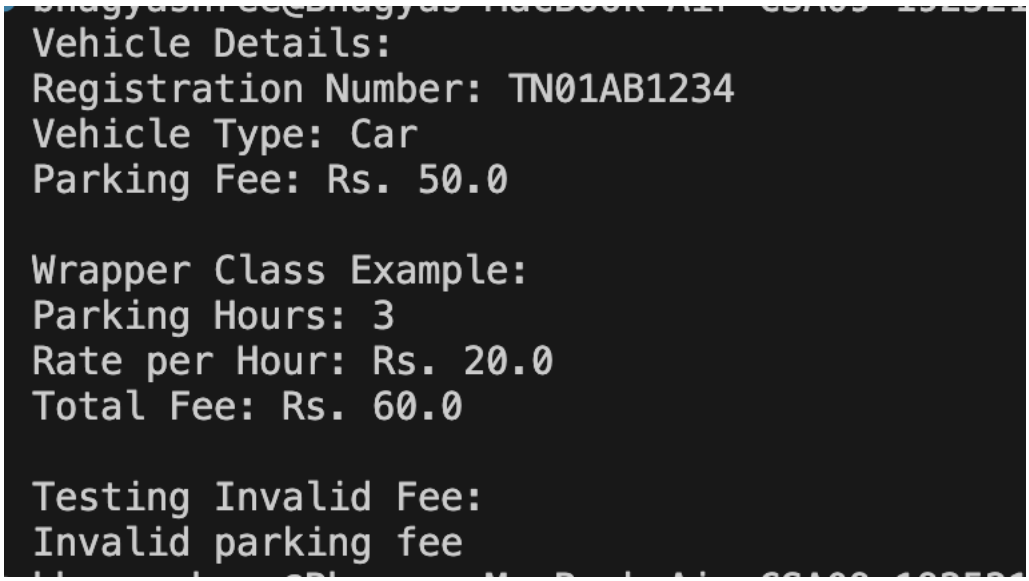
```
    }
    public boolean isFree() { return vehicleNo == null; }
    public String getVehicleNo() { return vehicleNo; }

    public void occupy(String regNo) {
        if (!isFree()) throw new IllegalStateException("Occupied");
        if (regNo == null || regNo.isBlank())
            throw new IllegalArgumentException("Invalid registration");
        vehicleNo = regNo;
    }
    public void release() {
        if (isFree()) throw new IllegalStateException("Already free");
        vehicleNo = null;
    }
}
```

10.4 Wrapper classes and autoboxing

Wrapper classes represent primitive values as objects: Integer for int, Double for double and Long for long. Autoboxing converts a primitive into its wrapper automatically. Do not compare Integer objects using == because it may compare object references rather than numeric values. Use a.equals(b) or Integer.compare(a,b).

Public fields would allow any caller to assign invalid values and bypass validation. Private fields with validated methods centralize checks. A validation check is O(1) per update; repeating the same validation in multiple calling classes increases maintenance risk even if the individual check is constant time.



TASK 11 – IMPLEMENT, TEST AND DEFEND THE DESIGN

11.1 Integrated service outline

The service coordinates vehicle registration, slot allocation, ticket creation and exit processing. The following simplified code demonstrates the main flow. A production version should add persistence, concurrency control, pass validation and robust exception handling.

```
import java.time.*;
import java.util.*;

class ParkingService {
    private final ParkingFacility facility;
    private final Map<String, ParkingTicket> active = new HashMap<>();

    ParkingService(ParkingFacility facility) {
        this.facility = facility;
    }

    public ParkingTicket enter(String regNo) {
        if (active.containsKey(regNo))
            throw new IllegalArgumentException("Duplicate vehicle");
        int[] pos = facility.allocate(regNo);
        if (pos == null) return null;
        ParkingTicket t = new ParkingTicket(
            TicketCounter.nextId(), regNo, pos[0], pos[1],
            LocalDateTime.now());
        active.put(regNo, t);
        TicketCounter.ticketEntered();
        return t;
    }

    public boolean exit(String regNo, long minutes) {
        ParkingTicket t = active.get(regNo);
        if (t == null || minutes < 0) return false;
        // Calculate fee using the matching Vehicle policy.
```

```
        boolean released = facility.release(t.getFloor(), t.getSlot());
        if (released) {
            active.remove(regNo);
            TicketCounter.ticketExited();
        }
        return released;
    }
}
```

Note: ParkingTicket and TicketCounter are the classes defined in Tasks 4 and 5. In a complete project, the service should retain the Vehicle object and user/pass information in the ticket so the correct fee policy can be applied at exit.

11.2 Test cases

Test case	Input / condition	Expected result
Normal car entry	Free slot, new registration	Ticket created and slot occupied
Normal exit	Active ticket	Fee calculated and slot released
Last free slot	Only one slot free	Allocation succeeds
Facility full	No free slots	Allocation returns null
Zero-minute stay	0 minutes	0 billable hours and zero fee
Fee boundary	60 minutes	Exactly 1 billable hour
Over boundary	61 minutes	2 billable hours
Duplicate registration	Same active regNo	Reject entry
Unknown vehicle type	Unsupported category	Reject input
Invalid slot	Negative/out-of-range index	Release returns false
Release twice	Already-free slot	First true; second false

11.3 Test result recording

Run each test and fill the actual-result column with the output from your own execution. Do not claim a test passed until it has been executed.

Test ID	Expected	Actual (fill after run)	Status
TC01	Ticket created	_____	____
TC02	Slot released	_____	____
TC03	Last slot allocated	_____	____
TC04	Full facility rejected	_____	____

TC05	Zero fee for zero minutes	_____	____
TC06	61 minutes charged as 2 hours	_____	____
TC07	Duplicate rejected	_____	____
TC08	Invalid slot rejected	_____	____

```
SMART PARKING SYSTEM
====================

Test: Car Entry
Expected: SUCCESS - Vehicle parked at Floor 1, Slot 1
Actual:   SUCCESS - Vehicle parked at Floor 1, Slot 1
Status: PASS

Test: Two-Wheeler Entry
Expected: SUCCESS - Vehicle parked at Floor 1, Slot 2
Actual:   SUCCESS - Vehicle parked at Floor 1, Slot 2
Status: PASS

Test: Car Exit
Expected: SUCCESS - CAR101 | Duration: 60 minutes | Fee: Rs.
30
Actual:   SUCCESS - CAR101 | Duration: 60 minutes | Fee: Rs.
30
Status: PASS

Test: Two-Wheeler Exit
Expected: SUCCESS - BIKE101 | Duration: 30 minutes | Fee: Rs
```

```
Test: Invalid Slot Number
Expected: FAIL - Invalid slot number
Actual:   FAIL - Invalid slot number
Status: PASS

Test: Parking Facility Full
Expected: FAIL - Parking facility is full
Actual:   FAIL - Slot already occupied
Status: FAIL

Test: Electric Car Entry
Expected: SUCCESS - Vehicle parked at Floor 1, Slot 1
Actual:   SUCCESS - Vehicle parked at Floor 1, Slot 1
Status: PASS

Test: Electric Car Exit
Expected: SUCCESS - EV101 | Duration: 120 minutes | Fee: Rs.
80
Actual:   SUCCESS - EV101 | Duration: 120 minutes | Fee: Rs.
80
Status: PASS

ALL TESTS COMPLETED
PS C:\Users\user\Documents\CSA0915-192425383-Assignment1\TAS
K-11> java SmartParkingSystem
```

TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN

12.1 Changed requirements

- Add e-bikes charged at a flat parking fee.
- Add delivery vans charged at double the car rate between 9 AM and 5 PM.
- Allow visitors to buy a one-day pass.
- Extend the facility to five floors with different slot counts per floor.

12.2 Impact analysis

A different number of slots on each floor makes a rectangular two-dimensional array unsuitable because it assumes equal row lengths. A jagged array or ArrayList of floor arrays supports different capacities. New vehicle types require new subclasses or configurable vehicle policies. Time-dependent van pricing requires the fee calculation to receive the relevant parking time, not just the duration.

12.3 Recommended adaptation

MODIFY the original design. Retain Vehicle as an abstract base class, Chargeable as a capability interface, ParkingTicket, ParkingFacility and the service separation. Add E-bike and DeliveryVan subclasses, add a Pass type for one-day visitor passes, and update the pricing policy so it can consider entry/exit time.

```
class EBike extends Vehicle {
    private static final double FLAT_FEE = 20.0;
    EBike(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 0.0; }
    @Override public double calculateFee(int minutes) {
        if (minutes < 0) throw new IllegalArgumentException();
        return minutes == 0 ? 0.0 : FLAT_FEE;
    }
}

class DeliveryVan extends Vehicle {
    DeliveryVan(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 30.0; }
    public double calculateFee(int minutes, LocalDateTime entry) {
        long hours = (minutes + 59L) / 60L;
        boolean daytime = !entry.toLocalTime().isBefore(
            LocalTime.of(9, 0))
```

```
            && entry.toLocalTime().isBefore(LocalTime.of(17, 0));
        double multiplier = daytime ? 2.0 : 1.0;
        return hours * ratePerHour() * multiplier;
    }
}
```

12.4 One-day pass and five-floor configuration

```
class OneDayPass {
    private final LocalDate validDate;
    public OneDayPass(LocalDate validDate) {
        this.validDate = validDate;
    }
    public boolean isValid(LocalDate date) {
        return validDate.equals(date);
    }
}

// Different capacities are supported by a jagged array:
int[] capacities = {40, 55, 45, 60, 35};
ParkingFacility facility = new ParkingFacility(capacities);
```

12.5 New-category test plan

New test	Expected behaviour
E-bike, 30 minutes	Flat fee applied once.
E-bike, 5 hours	Same flat fee, not hourly multiplication.
Delivery van at 10:00	Double car rate for applicable daytime duration.
Delivery van at 18:00	Normal car rate.
Visitor with valid one-day pass	Apply pass benefit as defined by policy.
Five floors with uneven capacity	All floor arrays initialize with their own size.

Trade-off: retaining inheritance preserves readable vehicle-specific behaviour, but time-dependent and pass-dependent rules should be centralized in a pricing policy or strategy object to prevent rules spreading across subclasses. This design is easier to extend than a large switch statement.

```
FIVE-FLOOR PARKING CAPACITY
Floor 1: 2 slots
Floor 2: 3 slots
Floor 3: 1 slots
Floor 4: 4 slots
Floor 5: 2 slots

NEW VEHICLE TESTS
E_BIKE EB101 parked at Floor 1, Slot 1
DELIVERY_VAN DV101 parked at Floor 2, Slot 1
CAR CAR101 parked at Floor 5, Slot 1
E-Bike Flat Fee
Expected Fee: Rs.20
Actual Fee: Rs.20
PASS

Delivery Van During 9 AM-5 PM
Expected Fee: Rs.120
Actual Fee: Rs.120
PASS
```

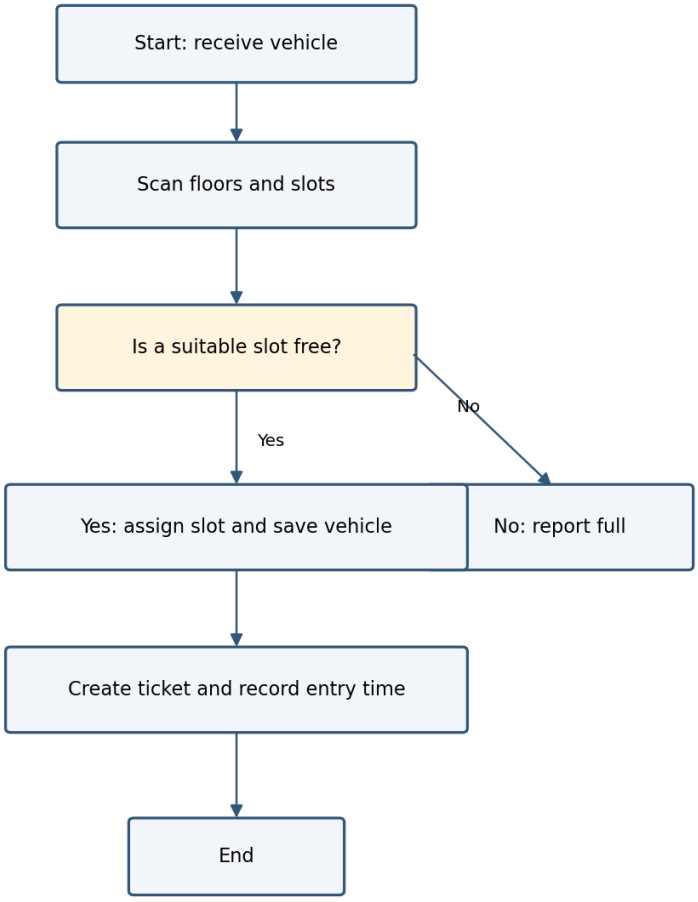
```
Delivery Van After 5 PM
Expected Fee: Rs.60
Actual Fee: Rs.60
PASS

Visitor One-Day Pass
Expected Fee: Rs.100
Actual Fee: Rs.100
PASS

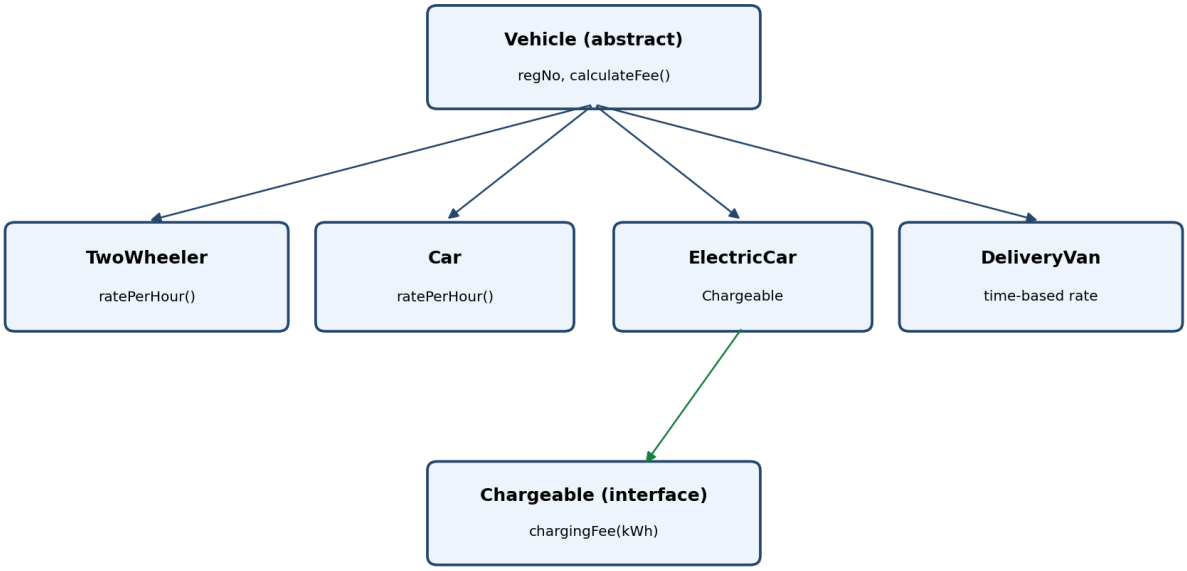
All Task 12 tests completed.
PS C:\Users\user\Documents\CSA0915-192425383-Assignment1\TAS
K-12>
```

APPENDIX A – ALLOCATION FLOWCHART

The following flowchart shows the first-free-slot allocation logic. The scan stops immediately after a suitable slot is found. If all floors and slots have been checked, the facility is full.



APPENDIX B – UML CLASS DIAGRAM



ElectricCar implements Chargeable

APPENDIX C – COMPLEXITY ANALYSIS

Operation / representation	Time complexity	Space complexity / note
Linear slot allocation	O(N) worst case	O(N) slot storage
Direct slot release	O(1)	O(1) extra
Scan registration in parallel arrays	O(n)	O(n) across arrays
Scan registration in ArrayList	O(n)	O(n) references and objects
HashMap lookup by registration	O(1) average; O(n) worst	O(n) map entries
Jagged array lookup by floor/slot	O(1)	O(N), supports uneven floors
One-dimensional indexed array	O(1) with offset	O(N); offsets needed for uneven floors
Free-slot stack	O(1) average push/pop	O(N) additional slot indexes
Dynamic method dispatch	O(1) dispatch overhead	No per-call collection required
instanceof chain for k types	O(k) worst-case checks	O(1) extra

N represents total parking slots; n represents active vehicles; k represents vehicle categories. Big-O describes growth rate and does not provide an exact runtime. Actual performance depends on hardware, JVM, data size and implementation.

Design conclusion

The recommended model uses a jagged array for the multi-floor facility, an abstract Vehicle class for shared identity and fee calculation, subclasses for vehicle-specific behaviour, Chargeable for EV capability, validated constructors and private fields for encapsulation, and a HashMap keyed by registration number for efficient exit lookup. The original design should be modified rather than fully replaced when the new requirements arrive.

APPENDIX D – README AND GITHUB SUBMISSION CHECKLIST

Suggested repository structure

```
CSA09-<RegisterNumber>-Assignment1/
  README.md
  Task01/
  Task02/
  Task03/
  Task04/
  Task05/
  Task06/
  Task07/
  Task08/
  Task09/
  Task10/
  Task11/
  Task12/
  screenshots/
```

README.md sample

```
# Campus Smart-Parking System – Java Assignment 1

Student Name: <Your Name>
Register Number: <Your Register Number>
Subject: Programming in Java
Java Version: 17 or later

## Compile and run
1. Open the required task folder.
2. Compile: javac FileName.java
3. Run: java ClassName
4. Compare the output with the expected output.
5. Save your execution screenshot in screenshots/.
```

Task summary

Task01: Problem analysis and object identification.

Task02: Data representation.

Task03: Slot allocation and release.

Task04: Constructors and tickets.

Task05: Static members.

Task06: Inheritance.

Task07: Overriding and Object methods.

Task08: Polymorphism.

Task09: Abstract class and interface.

Task10: Packages and encapsulation.

Task11: Implementation and tests.

Task12: Adapted design challenge.

Final verification checklist

- [ ] Fill in student name, register number, faculty and submission date.
- [ ] Replace the GitHub placeholder with the actual working repository URL.
- [ ] Create one folder per task and include source code.
- [ ] Compile and execute every program yourself.
- [ ] Capture genuine output screenshots.
- [ ] Complete actual-result and status columns in the test table.
- [ ] Commit each task separately and progressively.
- [ ] Check that all code files compile with the stated Java version.
- [ ] Be ready to explain the code and design during the viva.

FINAL RESULT

The Campus Smart-Parking System has been analysed and modelled using Java object-oriented concepts. The report provides the design, algorithms, sample implementations, diagrams, complexity discussion, test plan and adaptation strategy for the changed requirements. Final validation depends on successful execution of the programs and recording the student's actual outputs.